
Reinforcement Learning Assignment 2: REINFORCE and basic Actor-Critic Methods Report

Yutao Liu(s4213211)¹ Nael Belhaj Haddou (s4431278)¹ SISAY TADESSE KETEMA s4184343¹

Abstract

We explore and compare three policy gradient methods: REINFORCE, Actor-Critic, and Advantage Actor-Critic—using the CartPole-v1 environment. REINFORCE shows fast learning but high variance, A2C offers stable performance, and AC is slower and less stable. Results highlight the trade-offs between convergence speed and stability in policy-based reinforcement learning.

1. Introduction

In Assignment 1, we explored and demonstrated that value-based methods such as Q-learning are effective, but they suffer from the drawback of relying on implicitly derived policies from the value function. This assignment requires us to explore policy gradient methods: methods that directly parameterize the policy itself, which offers several potential advantages, including better handling of continuous action spaces and stochastic policies.

We are required to implement three key policy gradient algorithms in the CartPole environment: REINFORCE, a Monte Carlo policy gradient method; basic Actor-Critic (AC), which combines policy optimization with value function approximation; and Advantage Actor-Critic (A2C), which uses an advantage function to reduce variance in updates. These methods represent an evolution of policy-based methods, from pure policy optimization to hybrid methods that leverage both policy and value networks.

Finally, by comparing the performance of these algorithms to previous Q-learning implementations, we are able to analyze their convergence properties and understand the theoretical foundations that make each method effective. By systematically studying these methods, we seek to gain a deeper understanding of the strengths and limitations of policy gradient methods in reinforcement learning, which is what we gain from this assignment.

2. Theory

2.1. Vanilla Policy Loss

In value-based reinforcement learning, the goal is to find a policy that maximizes expected return. In the policy gradient approach, instead of estimating value functions to derive the policy, we directly optimize a parameterized policy $\pi_\theta(a|s)$ using gradient ascent.

The expected return of the policy is defined as:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T r_t \right]$$

where the θ are the parameters of the policy network, τ denotes a trajectory $(s_0, a_0, r_0, \dots, s_T)$ sampled from π_θ , and r_t is the reward at timestep t .

By the policy gradient theorem (Sutton & Barto, 2018), the gradient of $J(\theta)$ is:

$$\nabla_\theta J(\theta) = \mathbb{E}_{s \sim d^\pi, a \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a|s) Q^\pi(s, a)]$$

where d^π is the discounted state visitation distribution under policy π , and $Q^\pi(s, a)$ is the expected return when taking action a in state s and following π thereafter.

To enable gradient descent optimization, we define the **vanilla policy loss** as:

$$\mathcal{L}(\theta) = -\mathbb{E}_{s \sim D, a \sim \pi_\theta} [\log \pi_\theta(a|s) Q^\pi(s, a)]$$

The negative sign is added because optimizers typically perform gradient descent. D denotes the empirical distribution of states collected during training. Since $Q^\pi(s, a)$ is not known, it is estimated via Monte Carlo returns. This leads to the REINFORCE algorithm, where the above is referred to as the *vanilla policy loss*.

2.2. REINFORCE algorithm

The REINFORCE algorithm is a policy gradient method that directly optimizes the policy to maximize the expected

cumulative reward. It consists of the following components:

- **Policy Network:** $\pi_\theta(a|s)$ maps state s to a probability distribution over actions a , parameterized by θ . This network represents the policy that is being optimized.

The learning procedure includes:

- Collecting an episode of transitions (s_t, a_t, r_t) by interacting with the environment using the current policy π_θ .
- Computing the return for each time step t :

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

where r_{t+1} is the reward at time $t + 1$, and $\gamma \in [0, 1]$ is the discount factor.

- Updating the policy parameters θ to maximize the expected return using the policy gradient:

$$\mathcal{L}_{\text{policy}} = - \sum_t G_t \cdot \log \pi_\theta(a_t|s_t)$$

Here, $\log \pi_\theta(a_t|s_t)$ is the log-probability of the action a_t taken at state s_t , and G_t is the return from time step t .

- Performing gradient ascent on the policy loss to update the policy parameters:

$$\theta \leftarrow \theta + \alpha \cdot \nabla_\theta \mathcal{L}_{\text{policy}}$$

where α is the learning rate.

This process is repeated over multiple episodes until the policy converges or achieves satisfactory performance.

2.3. Actor-Critic Algorithm

2.3.1. AC ARCHITECTURE

The Actor-Critic (AC) algorithm consists of two networks:

- **Policy Network (Actor):** $\pi_\theta(a|s)$ maps state s to a probability distribution over actions a , parameterized by θ . Called the actor because it decides the actions taken.
- **Value Network (Critic):** $V_\phi(s)$ estimates the expected return from state s under the current policy, with parameters ϕ . Called the critic because it evaluates the actions taken.

The learning procedure including:

- Collecting transitions (s_t, a_t, r_t, s_{t+1}) through interaction with the environment at time step t .
- Computing the temporal difference (TD) target:

$$y_t = r_t + \gamma V_\phi(s_{t+1})$$

where r_t is the reward at time t , $V_\phi(s_{t+1})$ is the estimated value of the next state, and $\gamma \in [0, 1]$ is the discount factor.

- Updating the Critic by minimizing the squared TD error:

$$\mathcal{L}_{\text{critic}} = (V_\phi(s_t) - y_t)^2$$

2.3.2. Q-LEARNING VS. ACTOR-CRITIC

Q-learning is an **off-policy** method relying on $\max_a Q(s, a)$ to directly estimate optimal action values, which can introduce overestimation bias. However, Actor-Critic is typically **on-policy**, using the Critic to evaluate the current policy π_θ without seeking optimal actions explicitly.

2.3.3. LOSS FUNCTIONS

In our AC implementation:

- **Critic Loss:**

$$\mathcal{L}_{\text{critic}} = (r + \gamma V_\phi(s') - V_\phi(s))^2$$

- **Actor Loss:**

$$\mathcal{L}_{\text{actor}} = - \log \pi_\theta(a|s) \cdot \hat{Q}(s_t, a_t)$$

where $\hat{Q}(s_t, a_t)$ is the estimated Q value for action a in state s at timestep t .

Algorithm 1 Basic Actor-Critic (AC) Algorithm

Input: actor $\pi_\theta(a|s)$, critic $V_\phi(s)$

Parameters: discount factor γ

for each environment step **do**

 Observe state s , select action $a \sim \pi_\theta(a|s)$

 Execute action a , observe reward r , next state s' , and done flag d

 Compute TD target: $y = r + \gamma(1 - d) \cdot V_\phi(s')$

 Compute TD error: $\delta = y - V_\phi(s)$

 Update critic by minimizing loss: $\mathcal{L}_c = \delta^2$

 Update actor by minimizing: $\mathcal{L}_a = - \log \pi_\theta(a|s) \cdot \delta$

if d is true **then**

 Reset environment

end if

end for

2.4. A2C Algorithm

Advantage actor critic (A2C) is very similar to actor critic, it is mentioned in (Sutton & Barto, 2018). The main difference between the two is the use of an advantage function. The advantage is obtained by subtracting the state value from the Q value of the action. This creates values that are more centered around 0, helping the algorithm’s stability. These values serve the same purpose as the Q values, giving a value to an action in a given state. Advantage works as a proxy for the Q function in loss because it is calculated by subtracting from the Q values, creating the same results, simply with a scaling, in a given state if an action has a better Q value than another it will also have a better advantage.

The loss function used for the critic is the same as in vanilla actor critic, the subtraction can be replaced by the advantage function as their equation is the same. The actor loss is a little different, instead of multiplying the log probabilities by the Q values, they are multiplied by the advantage values. This creates the loss functions used in the A2C, showcased in 2. Where a_t , s_t , r_t are the action, the state and the reward reward at time step t respectively.

Algorithm 2 Advantage Actor-Critic (A2C) Algorithm

Input: actor $\pi_\theta(a|s)$, critic $V_\phi(s)$, environment

Parameters: discount γ , learning rate α , episode per update M , estimation depth n

sample M episode traces from environment following policy π_θ

for each trace do

for each environment step t in trace do

$$\hat{Q}_n(s_t, a_t) \leftarrow \sum_{k=0}^{n-1} \gamma^k \cdot r_{t+k} + \gamma^n \cdot r_n$$

$$\hat{A}_n(a_t, s_t) \leftarrow \hat{Q}_n(s_t, a_t) - V_\phi(s_t)$$

end for

end for

$$\phi \leftarrow \phi - \alpha \cdot \nabla_\phi \sum_t (\hat{A}_n(s_t, a_t))^2$$

$$\theta \leftarrow \theta + \alpha \cdot \sum_t [\hat{A}_n(s_t, a_t) \cdot \nabla_\theta \log \pi_\theta(a_t|s_t)]$$

3. Experiment

3.1. Environment

The cart-pole environment is used in all experiments described here, it tasks an agent with balancing a pole on top of a cart by moving it left and right. Episodes have the agent start with the pole at an angle of 0 and the cart at a position in the range ± 0.05 and with both not moving. Episodes terminate when the cart gets more than 2.4 units away from the center, when the angle of the pole gets out of the range $\pm 12^\circ$ or after 500 steps. The agent gets a reward of 1 for every step. Observations are arrays containing 4 values: the cart’s position and velocity and the pole’s angle and angular velocity. The action space is one discrete number taking the

values 0 or 1 to push the cart left or right respectively.

3.2. REINFORCE Algorithm

We implemented the REINFORCE algorithm to train a policy network directly using Monte Carlo return estimates. The agent interacts with the CartPole-v1 environment and updates the policy parameters after each completed trajectory. Unlike value-based methods, our implementation doesn’t use a critic network or temporal-difference bootstrapping. Instead, the policy is trained using the vanilla policy gradient objective derived from the policy gradient theorem.

Figure 1 shows the REINFORCE learning curve. The agent achieves steady improvement, reaching average returns above 400 within 100K steps. However, we observed relatively large standard deviations across runs, especially from the middle training to the end, reflecting the high variance nature of REINFORCE without a baseline.

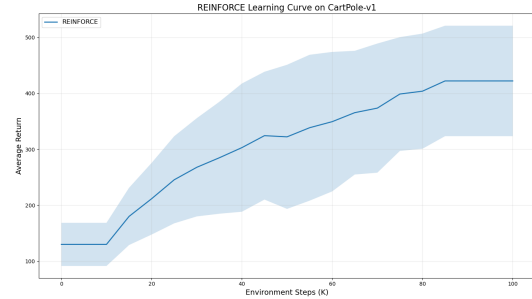


Figure 1. Learning curve of the REINFORCE agent on CartPole-v1, averaged over 5 independent runs using 4 parallel environments. The solid line indicates the smoothed average return and the shaded region represents one standard deviation. Despite high variance, the agent consistently improves and achieves near-optimal performance Using NVIDIA GeForce RTX 4060 GPU for speeding training process.

3.3. AC Algorithm

For our basic Actor-Critic implementation, we used separate neural networks for the actor and critic. The actor network outputs a probability distribution over actions, while the critic estimates the state value function $V(s)$. The actor is updated using TD error as a surrogate for the advantage. We performed training using 8 parallel environments with batched updates for improved efficiency.

The learning curve of our basic Actor-Critic agent is shown in Figure 2. The agent demonstrated consistent learning progress, with average scores improving steadily from around 150 to 290 over 100K environment steps. While the results show some variance between runs, the overall performance remains stable without collapsing, suggesting

our implementation effectively balances exploration and exploitation.

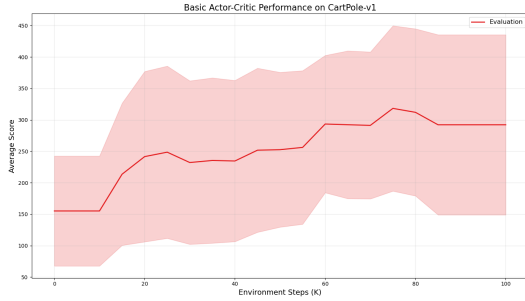


Figure 2. Learning curve of the basic Actor-Critic agent on CartPole-v1, averaged over 5 runs with 8 parallel environments. The solid line represents the smoothed average return; the shaded area indicates standard deviation. Performance increases steadily and stabilizes around 290, with moderate variance across runs. Using NVIDIA GeForce RTX 4060 GPU for speeding training process.

3.4. A2C Algorithm

The A2C algorithm is trained in the cart-pole environment for 1 million steps and evaluated every 5 thousand steps, repeated 5 times. Both actor and critic use a learning rate of 0.0007 and are separate neural networks with 2 hidden layers with 64 units each. Because of exploding gradients issues, gradients are clipped to have norm up to 0.5. A discount factor of 0.99 is used. An estimation depth of 5 steps is used and backpropagation is performed after every run. Results are showcased in figure 3.

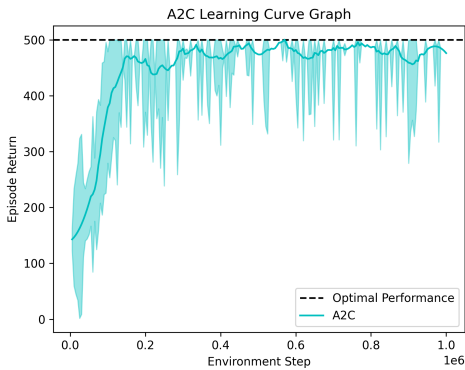


Figure 3. Learning curve Advantage Actor Critic over 5 runs of 1 million steps of training in the cart-pole environment. Results are averaged and smoothed. Optimal performance is reached after about 175 thousand steps. Standard deviation is quite small, indicating stability.

The algorithm converges relatively quickly, reaching opti-

mal performance in under 200 thousand steps. It performs quite stably, having an overall small standard deviation and no large drops in performance at any point.

3.5. Unified Experiment

All the algorithms presented above are compared to each other and to a deep Q network (DQN) over 5 runs of 1 million steps each. Every 5 thousand steps the agents are evaluated, the results can be seen in figure 4. All agents use the same hyperparameters as in their individual sections except AC. AC's learning rate is changed to 0.00005 for the actor and 0.0001 for the critic. The DQN uses experience replay and a target network. Its network has 2 hidden layers of 128 units each. It uses a gamma of 0.99 and an epsilon-greedy strategy with an epsilon decaying epsilon starting at 0.9. The network is updated every 10 steps and the target network is updated every 200 steps.

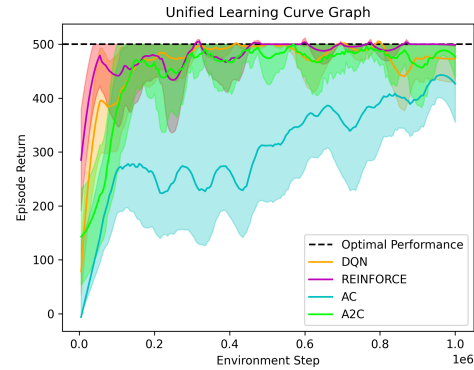


Figure 4. Learning curves of DQN, REINFORCE, Actor Critic and Advantage Actor Critic over 5 runs of 1 million steps of training in the cart-pole environment. REINFORCE converges to optimal performance the fastest and performs stably throughout training. AC converges the slowest and has the largest standard deviation, indicating some instability. DQN and A2C both reach optimal performance a little after REINFORCE and perform relatively stably.

While all the agents eventually reach optimal performance in at least one run, actor critic is the slowest by far. On the other hand, REINFORCE converges very quickly and reaches optimality within 100 thousand steps. Actor critic also appears to be the least stable agent, having the widest standard deviation. A2C appears to be the stablest agent, having the smallest standard deviation overall. Although, REINFORCE is quite close behind, as is DQN.

Discussion

Because of how simple the environment is, it makes sense to see good performances from all agents. The performance of

REINFORCE and DQN compare to the actor critic models is surprising as they are supposed to have more reliable convergence. An explanation for this may be the gradient clipping which slowed their convergence.

A2C performs far better than AC, this is expected as the baseline subtraction is introduced in part to aid numerical stability. This stability along with a smaller gradient appears to have promoted a much faster convergence and higher stability.

An ablation study of each agent type's hyperparameters would give greater insight into what allows each to perform well. This could allow for a fairer comparison of the algorithms as it would allow to compare them all at their best and to include their sensitivity to hyperparameter settings in the discussion. Experiments using more challenging environments would allow for better comparison of the agents' robustness and capabilities.

Conclusion

The comparison of DQN, REINFORCE, actor critic and advantage actor critic presented here shows that all these agents are able to solve the cart-pole environment. REINFORCE and A2C appear to perform the best on this environment and are both quite stable, getting similar results in most runs. Actor critic is very slow to converge, likely because of its gradient being clipped but the issues with exploding gradients puts in question its numerical stability. Further experiments on different hyperparameter settings and environments would yield interesting conclusions.

Limitation

Although our experiments effectively reveal the performance comparison of DQN, REINFORCE, Actor Critic and Advantage Actor Critic in the Cart-Pole environment, there are still some limitations in the experiments:

First, the gradient clipping (norm up to 0.5) used to solve the gradient explosion problem in A2C may affect the convergence speed. Alternative gradient stabilization techniques may produce different performance characteristics.

Second, as part of the experimental requirements, we only performed the most basic implementation of all algorithms without optimizing, for example, the network architecture. All implementations used relatively simple neural network architectures, lacking exploration of the impact of more complex architectures on performance

Finally, the CartPole environment, while useful for benchmarking, is relatively simple, with a low-dimensional state space (4 features) and a binary action space. Testing only on CartPole limits our ability to compare algorithm robustness

across different RL problem domains, especially those with continuous action spaces, where policy gradient methods have theoretical advantages.

Addressing these limitations in our future work would provide more comprehensive insights into the relative strengths and weaknesses of these policy gradient algorithms.

References

Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018. ISBN 9780262039246. URL <http://incompleteideas.net/book/the-book-2nd.html>.